ỦY BAN NHÂN DÂN
THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

ĐỀ CƯƠNG TRÍ TUỆ NHÂN TẠO NÂNG CAO

I – Phần thông tin chung

Họ và tên sinh viên 1: Nguyễn Tuyết Nhi
MSSV : 3122410282 Lớp: DCT1223
E-mail : nhituyet20042008@gmail.com
Ngành : Công nghệ thông tin
Chuyên ngành : Khoa học máy tính

Họ và tên sinh viên 2: Trần Tấn Đạt
MSSV : 3122410078 Lớp:DCT1222
E-mail : trandattb04@gmail.co
Ngành : Công nghệ thông tin
Chuyên ngành : Khoa học máy tính

Họ và tên sinh viên 3: Trần Ngọc Ánh
MSSV : 3122410013 Lớp: DCT1221
E-mail :anhanh41214013@gmail.com
Ngành : Công nghệ thông tin
Chuyên ngành : Khoa học máy tính

Họ và tên sinh viên 4: H’ Như Lưk
MSSV : 3122410226 Lớp:DCT1222
E-mail : nhubkr1010@gmail.com
Ngành : Công nghệ thông tin
Chuyên ngành : Khoa học máy tính

Tên đề tài: XÂY DỰNG MÔI TRƯỜNG VÀ TÍCH HỢP THUẬT TOÁN TÌM KIẾM TRONG TRÒ CHƠI CỜ CARO (11 x 11)

II - Nội dung đề cương

1. Lý do chọn đề tài

Tính thách thức và mở rộng: Trò chơi Tic Tac Toe tiêu chuẩn (3 x 3) quá đơn giản và có thể giải quyết triệt để dễ dàng. Việc mở rộng kích thước bàn cờ lên 11 x 11 (với điều kiện thắng là nối liền 5 quân cờ, tương tự luật chơi Gomoku/Caro) làm bùng nổ không gian tìm kiếm, đòi hỏi AI phải sử dụng các chiến lược thông minh thay vì vét cạn.

Mục tiêu học tập: Vận dụng sâu sắc các kiến thức về không gian trạng thái phức tạp, các thuật toán cắt tỉa (Alpha-Beta Pruning), kỹ thuật tìm kiếm có giới hạn độ sâu (Depth-Limited Search) và thiết kế hàm heuristic đánh giá thế cờ.

Ứng dụng thực tiễn: Xây dựng một môi trường mô phỏng trò chơi chuẩn hóa với không gian hành động lớn (121 ô), phục vụ cho việc kiểm thử các mô hình tác nhân thông minh (AI Agents).

2. Mục đích và nhiệm vụ nghiên cứu

Mục tiêu chính:

- Thiết kế môi trường trò chơi Tic Tac Toe kích thước 11 x 11 với luật chơi nối 5 quân trên nền tảng Python.
- Nghiên cứu, thiết kế hàm heuristic đánh giá chuỗi (tấn công/phòng thủ như tạo hàng 3, hàng 4 mở) và tích hợp thuật toán tìm kiếm tối ưu để AI đưa ra nước đi hợp lý trong thời gian cho phép.

Nhiệm vụ cụ thể:

- Xây dựng cấu trúc biểu diễn bàn cờ 11 x 11 và các hàm kiểm tra điều kiện thắng/thua/hòa (kiểm tra hàng ngang, dọc, chéo).
- Cài đặt thuật toán tìm kiếm Minimax kết hợp cắt tỉa Alpha-Beta với độ sâu tìm kiếm thích hợp (depth = 2 hoặc 3) hoặc tích hợp thuật toán tìm kiếm cục bộ/heuristic do giới hạn thời gian tính toán.
- Xây dựng giao diện trực quan (hoặc qua Console/Streamlit) để người chơi tương tác đấu với AI.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Không gian trạng thái lớn, các kỹ thuật tối ưu hóa tìm kiếm đường đi và hàm đánh giá heuristic trong trò chơi bàn cờ kích thước lớn.

Phạm vi nghiên cứu:

- Bàn cờ giới hạn ở kích thước 11 x 11.
- Luật chơi: Người chơi đầu tiên nối liên tiếp 5 quân cờ (ngang, dọc hoặc chéo) trước sẽ giành chiến thắng.

4. Phương pháp nghiên cứu

Phương pháp lý thuyết: Nghiên cứu lý thuyết trò chơi (Game Theory), lý thuyết không gian trạng thái và các kỹ thuật cắt tỉa không gian tìm kiếm.

Phương pháp thực nghiệm: Lập trình hiện thực hóa mã nguồn Python, tinh chỉnh trọng số của hàm heuristic, đo lường thời gian phản hồi (response time) của AI và thử nghiệm đấu thực tế để đánh giá độ thông minh.

5. Ý nghĩa khoa học và ý nghĩa thực tiễn

Phương pháp lý thuyết: Nghiên cứu lý thuyết trò chơi (Game Theory), lý thuyết không gian trạng thái và các kỹ thuật cắt tỉa không gian tìm kiếm.

Phương pháp thực nghiệm: Lập trình hiện thực hóa mã nguồn Python, tinh chỉnh trọng số của hàm heuristic, đo lường thời gian phản hồi (response time) của AI và thử nghiệm đấu thực tế để đánh giá độ thông minh.

6. Dự kiến kế hoạch thực hiện sơ bộ

Tuần	Công việc dự kiến	Ghi chú
1	Xây dựng logic cốt lõi cho bàn cờ 11 x 11, luật kiểm tra thắng thua cho 5 quân cờ liên tiếp.	
2	Thiết kế và cài đặt hàm đánh giá heuristic (nhận diện các thế cờ nguy hiểm, các hàng chờ tấn công).	
3	Tích hợp thuật toán Minimax kết hợp Alpha-Beta Pruning và tối ưu hóa hiệu năng tính toán.	
4	Xây dựng giao diện thử nghiệm, cho AI tự đấu với người hoặc tự đấu với chính nó để tinh chỉnh tham số.	
5	Hoàn thiện báo cáo tổng kết và chuẩn bị tài liệu nộp.	

7. Dự kiến nội dung của đề tài

7.1 Tìm hiểu, xây dựng và kiểm tra các chiến lược AI dựa trên luật trò chơi

Phần này tìm hiểu về việc xây dựng môi trường game Tic Tac Toe và lập trình các quy tắc AI tìm kiếm, nhìn trước nước đi đơn giản.

7.1.1 Tạo môi trường cho Tic Tac Toe game

a) Các phương thức và thuộc tính:

Định nghĩa các thuộc tính như: action_space, observation_space, state, reward, done và phương thức: reset, step, render, sample của môi trường game Tic Tac Toe.

- action_space: Tập hợp các vị trí có thể đánh trên bàn cờ 11×11.
- observation_space: Cấu trúc mảng và trạng thái đại diện cho bàn cờ hiện tại.
- state: Trạng thái hiện tại của ván đấu.
- reward: Giá trị thưởng phạt thắng, thua, hòa hoặc chưa kết thúc.
- done: Cờ hiệu xác định ván cờ đã kết thúc hay chưa.

Các phương thức chính: reset() - khởi tạo lại ván mới, step(action) - thực hiện nước đi và trả về trạng thái mới, render() - hiển thị bàn cờ, sample() - chọn nước đi ngẫu nhiên.

b) Tạo và làm quen với môi trường trò chơi:

Khởi tạo môi trường, hiển thị bàn cờ và kiểm tra các không gian hành trạng thái.

c) Thử nghiệm cách chơi và chơi thử trò chơi:

Thực hành chơi thử game bằng cách chọn nước đi ngẫu nhiên hoặc nhập trực tiếp từ bàn phím.

7.1.2 Tình trước nước đi thắng trong trò chơi:

Tạo AI biết tính trước nước đi thắng: Tạo thuật toán AI duyệt qua tất cả các nước đi tiếp theo để kiểm tra xem có nước đi nào mang lại chiến thắng ngay lập tức hay không.

Thử nghiệm AI có khả năng tính trước không: Đánh giá hiệu năng của AI nhìn trước 1 nước thông qua việc cho thi đấu 100–1000 trận với agent đánh ngẫu nhiên (Random Agent) và người chơi thật, ghi nhận tỷ lệ thắng/hòa/thua.

7.1.3 Tình trước nước đi chặn đối thủ trong trò chơi:

a) **Tạo Agent tính trước nước đi chặn đối thủ:** Nâng cấp AI để kiểm tra và chặn nước đi chiến thắng của đối thủ ở lượt tiếp theo.
   - Kiểm tra xem có nước đi thắng ngay hay không (Look-Ahead 1 step).
   - Nếu không có, mô phỏng nước đi của đối thủ ở lượt tiếp theo để phát hiện và chặn các nguy cơ đối thủ thắng ngay.

b) **Thử nghiệm AI có khả năng tính trước không:** Thử nghiệm và so sánh tỷ lệ thắng - thua của AI nhìn trước 2 nước so với AI nhìn trước 1 nước.

7.1.4 Tình trước nước đi tấn công kép trong trò chơi:

- **Tạo Agent tính trước nước đi tấn công kép:** Lập trình AI tính toán 3 nước đi để tạo ra thế cờ tấn công kép (double attack) nhằm đảm bảo chiến thắng.
- **Thử nghiệm AI có khả năng tính trước không:** Kiểm tra hiệu quả của AI nhìn trước 3 nước khi đối đầu với các phiên bản AI trước đó.

7.2: Cắt tỉa cây tìm kiếm MiniMax để giảm số bước tìm kiếm

Phần này áp dụng thuật toán tìm kiếm cây MiniMax và kỹ thuật cắt tỉa theo chiều sâu (Depth Pruning).

7.2.1 Thuật toán MiniMax cho trò chơi:

- **Tạo thuật toán:** Lập trình thuật toán MiniMax đệ quy để duyệt toàn bộ cây trạng thái game Tic Tac Toe nhằm tìm nước đi tối ưu hoàn hảo.
- **Thử nghiệm:** Cho người chơi đấu trực tiếp với thuật toán MiniMax và đo thời gian tính toán nước đi.
- **Độ hiệu quả:** Đánh giá độ chính xác và khả năng giải hoàn toàn (solve) trò chơi Tic Tac Toe của MiniMax.

7.2.2 Cắt tỉa theo độ sâu của trò chơi:

Giới hạn độ sâu tìm kiếm của MiniMax để giảm thời gian tính toán nước đi.

7.3: Cắt tỉa Alpha-Beta

Cải tiến thuật toán MiniMax bằng kỹ thuật Cắt tỉa Alpha-Beta để loại bỏ các nhánh không cần thiết.

7.3.1 Cắt tỉa Alpha-Beta trong trò chơi:

- **Định nghĩa hàm maximized_payoff_ttt():** Định nghĩa hàm tính toán phần thưởng tối đa có áp dụng biến cắt tỉa alpha và beta.
- **Định nghĩa hàm MiniMax_ab() :** Xây dựng thuật toán MiniMax_ab kết hợp cắt tỉa Alpha-Beta cho Tic Tac Toe.

7.3.2. Hiệu suất của Alpha-Beta Pruning: Đo lường và so sánh tốc độ xử lý (tiết kiệm đến 97% thời gian) so với MiniMax thuần túy.

7.4: Monte Carlo Tree Search

Thử nghiệm MCTS Agent:

- **Chơi thử MCTS Agent:** Chơi thử nghiệm trực tiếp với agent MCTS.
- **Đánh giá MCTS Agent:** Đánh giá hiệu quả chọn nước đi của công thức UCT (Upper Confidence Bounds for Trees) trong MCTS.

7.5. Policy Networks

Ứng dụng Mạng nơ-ron cuộn (CNN) để huấn luyện Mạng chính sách (Policy Network) cho trò chơi.

7.5.1 Deep Learning:

- **Huấn luyện mạng nơ-ron:** Thực hiện các bước huấn luyện mạng nơ-ron cho trò chơi.
- **Generate Expert Moves:** Sử dụng thuật toán MiniMax (Alpha-Beta) từ 7.6 để tạo dữ liệu các nước đi chuyên gia (expert moves).

7.5.2 Hai mạng Policy:

- **Tạo hai mạng Neural Networks:** Xây dựng hai mạng nơ-ron (mạng chính sách nhanh và mạng chính sách mạnh) xử lý trạng thái bàn cờ dạng đa chiều.
- **Huấn luyện hai Policy Networks:** Huấn luyện hai mạng chính sách dự đoán nước đi chuyên gia dựa trên dữ liệu đã tạo.

7.5.3 Thuật toán Mixed MCTS:

- **Tăng cường UCT Formula với Policy Network mạnh:** Cải tiến công thức UCT bằng cách tích hợp dự đoán từ Mạng chính sách mạnh.
- **Mixed MCTS:** Xây dựng thuật toán MCTS hỗn hợp (Mixed MCTS) kết hợp giữa tìm kiếm cây và học sâu.

7.6: Tối ưu hóa vector để tăng tốc học tăng cường

Chương này áp dụng Học tăng cường sâu tự đấu (Self-Play Deep RL) và kỹ thuật Vector hóa để tăng tốc độ huấn luyện.

7.6.1 Xử lý các nước đi không hợp lệ:

Xử lý bài toán gán thưởng/phạt cho các nước đi không hợp lệ (illegal moves) khi số ô trống giảm dần theo lượt chơi.

7.6.2 Học tăng cường sâu thông qua tự chơi:

- **Kết hợp hai Networks thành một đầu vào:** Mã hóa trạng thái bàn cờ theo góc nhìn độc lập với người chơi (đổi góc nhìn 1 và -1).
- **Đấu với Strong Policy Agent:** Cho mô hình học tăng cường tự đấu với Mạng chính sách mạnh.
- **Huấn luyện người chơi X and O:** Áp dụng thuật toán Policy Gradient để huấn luyện đồng thời hai người chơi X và O.

7.6.3 Huấn luyện Value Network:

Xử lý dữ liệu trải nghiệm trận đấu và xây dựng Mạng giá trị (Value Network) để dự đoán kết quả trận đấu từ trạng thái bàn cờ hiện tại.

7.6.4 Thử chơi với Value Network:

Xây dựng chiến thuật dựa trên Mạng giá trị và kiểm tra hiệu quả khi đối đầu với các agent khác.

7.7: AlphaGo và Connect Four

Kết hợp MCTS với 3 mạng nơ-ron (Policy mạnh, Policy nhanh, Value) để tạo ra mô hình AlphaGo hoàn chỉnh.

Thử nghiệm AlphaGo Agent:

- **Opponent:** Thiết lập đối thủ thử nghiệm cho AlphaGo.
- **AlphaGo so với Rule-Based AI:** Cho AlphaGo thi đấu với Rule-Based AI hoàn hảo từ Chapter 6 (kết quả luôn hòa, chứng minh AlphaGo đã "giải" được game).

7.8: Tự chơi lặp đi lặp lại và AlphaZero

Xây dựng thuật toán AlphaZero (chỉ học từ tự đấu từ đầu - zero knowledge, không dựa vào dữ liệu người chơi).

- **Lập kế hoạch huấn luyện AlphaZero:** Lập kế hoạch các bước huấn luyện AlphaZero và khởi tạo mạng Policy Gradient với trọng số ngẫu nhiên.
- **Huấn luyện AlphaZero:** Thực hiện vòng lặp huấn luyện tự đấu lặp đi lặp lại (Iterative Self-Play), cập nhật trọng số mạng nơ-ron qua từng thế hệ cho đến khi đạt trình độ chơi hoàn hảo.
- **Thử nghiệm AlphaZero:** Đánh giá năng lực của thuật toán AlphaZero sau khi hoàn thành huấn luyện.

8. Danh mục tài liệu tham khảo

1. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. (Đặc biệt tham khảo Chương 5: Adversarial Search and Games về Minimax, Alpha-Beta Pruning và Evaluation Functions).

2. Silver, D., et al. (2017). Mastering the game of Go without human knowledge. Nature, 550(7676), 354–359. (Tài liệu gốc nền tảng về phương pháp tự đấu Self-Play và Deep Reinforcement Learning cho cờ).

3. Silver, D., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362(6419), 1140–1144. (Nền tảng cho cấu trúc AlphaZero áp dụng chung cho các trò chơi bàn cờ).

4. Kochenderfer, M. J., & Wheeler, T. A. (2019). Algorithms for Optimization. MIT Press. (Tham khảo các phương pháp tối ưu hóa hàm heuristic và không gian trạng thái).

5. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. (Tham khảo về Policy Gradient và thuật toán học tăng cường).

6. Browne, C. B., et al. (2012). A survey of Monte Carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in Games, 4(1), 1–43. (Tài liệu tổng quan về giải thuật tìm kiếm cây Monte Carlo - MCTS).